

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 黄静怡 学号 2024K8009907042 专业 计算机科学与技术
实验项目编号 4.1 实验名称 处理器性能增强设计

- 注 1: 撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 `/home/serve-ide/cod-lab/reports` 目录下(注意: reports 全部小写)。文件命名规则: `prjN.pdf`, 其中 `prj` 和后缀名 `pdf` 为小写, `N` 为 1 至 4 的阿拉伯数字。例如: `prj1.pdf`。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: `prj5-projectname.pdf`, 其中“-”为英文标点符号的短横线。文件命名举例: `prj5-dma.pdf`。具体要求详见实验项目 5 讲义。
- 注 2: 使用 `git add` 及 `git commit` 命令将实验报告 PDF 文件添加到本地仓库 master 分支, 并通过 `git push` 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。
- 注 3: 实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

完整实验流程包括设计、开发、调试三个步骤:

- 设计步骤为在代码编写前的准备阶段, 主要包括模块划分和模块结构设计, 并绘制设计图纸。
- 开发步骤为在 S-IDE 集成开发环境中进行 Verilog/C 代码编写, 并实现完整功能。
- 调试步骤包括本地仿真验证后推送至云平台进行 FPGA 原型验证, 以及后续迭代调试的全过程。

一、设计成果(对设计流程的总结。需附上设计结构图, 可用 PPT 或者在线绘图软件绘制逻辑结构框图后保存为 PDF, 再插入到 L^AT_EX 中)

由于流程图比较复杂, 在此处借助了 chatgpt 生成流程图图片如图一

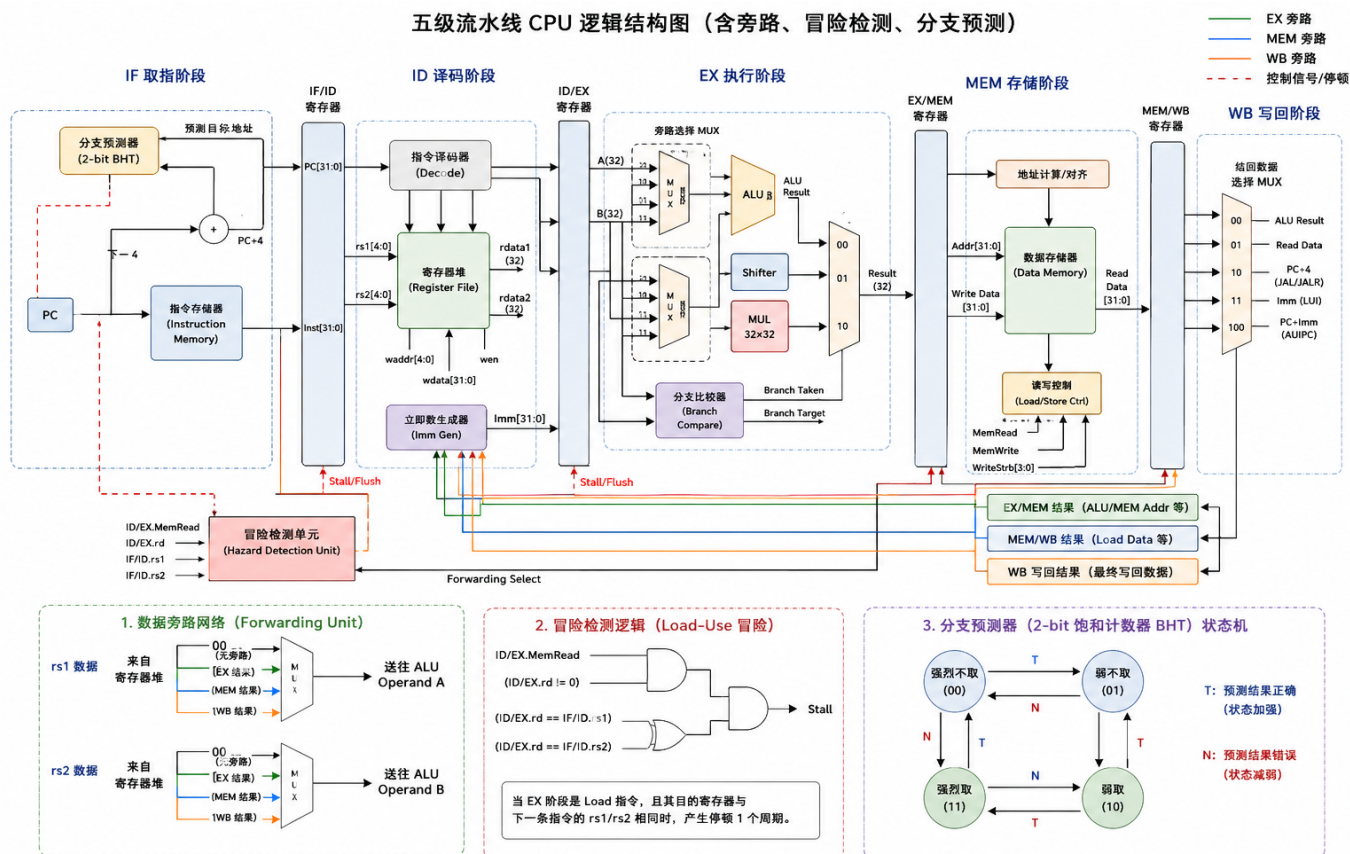
二、开发成果(对代码开发流程的总结。需附上包含注释的 VerilogHDL 关键代码段、相应信号的仿真波形和信号变化的说明)

1 流水线设计

本次是实验所采用的指令集是 RISC-V 指令集, 并且基于 prj3 的多周期 cpu 来进行流水线的适应性改造。在本次实验中, 我实现了 5 级 cpu 流水线, 包括 IF(取指), ID(译码), EX(执行), MEM(存储器访问), WB(写回)。流水线相邻阶段采用 Valid/Ready 握手机制实现流量控制, 核心握手信号:

```
wire if_to_id_valid; // IF→ID有效信号
wire id_ready;      // ID就绪信号
wire id_to_ex_valid; // ID→EX有效信号
wire ex_ready;      // EX就绪信号
wire ex_to_mem_valid; // EX→MEM有效信号
wire mem_ready;     // MEM就绪信号
wire mem_to_wb_valid; // MEM→WB有效信号
wire wb_ready;      // WB就绪信号
```

五级流水线 CPU 逻辑结构图 (含旁路、冒险检测、分支预测)



2 分支预测器

分支条件成立时,已进入流水线取指和译码阶段的指令将被无效(如前所述的 IF、IW 和 ID 级,造成 3 条无效指令),造成处理器性能损失,如能预先知道分支是否发生,会将正确的目标指令提前流入流水线,避免流水线被清空。

在本次实验中,我采用了四阶段分支预测器(强分支,弱分支,弱不分支,强不分支),当一次分支预测不正确之后,其分支预测当前阶段向另一侧移动一个阶段,其设计的主要目的是为了减少分支预测错误带来的流水线冲刷和阻塞,根据当前指令的分支预测结果逐渐调整预测的方向,使得分支预测器能够更好地适应当前指令的分支情况。

2.1 预测状态定义

```
localparam BP_INIT = 5'b00001; // 初始状态
localparam BP_S_TAKE = 5'b00010; // 强跳转
localparam BP_W_TAKE = 5'b00100; // 弱跳转
localparam BP_W_NTAK = 5'b01000; // 弱不跳转
localparam BP_S_NTAK = 5'b10000; // 强不跳转
reg [4:0] predictor_state; // 分支预测状态寄存器
```

2.2 预测逻辑与状态更新

区分条件分支与无条件跳转:无条件跳转直接判定为“跳转”,条件分支由两位计数器状态判定是否预测跳转。

分支目标地址计算:分别计算分支偏移地址、JAL/JALR 跳转地址,将预测结果与目标地址组合后传递至 IF 阶段用于 PC 预取。

状态更新逻辑:仅当分支指令进入执行阶段时,根据预测正误更新预测器状态:

```
always @(posedge clk) begin
    if (rst) begin
        predictor_state <= BP_INIT;
    end else if (id_to_ex_valid && branch_id) begin
        case (predictor_state)
            BP_INIT: predictor_state <= BP_S_TAKE;
            BP_S_TAKE: predictor_state <= prediction_incorrect ? BP_W_TAKE : BP_S_TAKE;
            BP_W_TAKE: predictor_state <= prediction_incorrect ? BP_W_NTAKES : BP_S_TAKE;
            BP_W_NTAKES: predictor_state <= prediction_incorrect ? BP_S_NTAKES : BP_W_TAKE;
            BP_S_NTAKES: predictor_state <= prediction_incorrect ? BP_S_NTAKES : BP_W_NTAKES;
            default: predictor_state <= BP_INIT;
        endcase
    end else if (!rst && predictor_state == BP_INIT) begin
        predictor_state <= BP_S_TAKE;
    end
end
```

2.3 预测错误检测与流水线冲刷

在 EX 阶段对比预测结果与分支真实执行结果,生成全局冲刷信号 prediction_incorrect:

```
wire ex_branch_real = ex_branch_check && ex_branch;
assign prediction_incorrect = (ex_branch_real ^ ex_prediction) && ~ex_block_cancel;
assign pc_correct = (ex_branch_real || ex_jump) ? ex_prediction_addr : (ex_pc + 32'd4);
```

3 IF 取指阶段设计

```
localparam IF_INIT = 5'b00001;
localparam IF_REQ = 5'b00010; // 发起指令请求
localparam IF_WAIT = 5'b00100; // 等待指令返回
localparam IF_SEND = 5'b01000; // 向ID阶段发送指令
localparam IF_ADVANCE = 5'b10000; // 更新PC
reg [4:0] if_state;
```

3.1 状态机跳转逻辑

所有状态跳转优先判断 prediction_incorrect,一旦预测错误立即回退至 IF_REQ,实现流水线冲刷;正常流程依次完成请求指令 → 等待响应 → 发送指令 → 更新 PC。

```
always @(*) begin
    case (if_state)
```

```

IF_INIT: if_next_state = IF_REQ;
IF_REQ: begin
    if (prediction_incorrect) if_next_state = IF_REQ;
    else if (Inst_Req_Ready && Inst_Req_Valid) if_next_state = IF_WAIT;
    else if_next_state = IF_REQ;
end
IF_WAIT: begin
    if (prediction_incorrect) if_next_state = IF_REQ;
    else if (Inst_Ready && Inst_Valid) if_next_state = IF_SEND;
    else if_next_state = IF_WAIT;
end
IF_SEND: begin
    if (prediction_incorrect) if_next_state = IF_REQ;
    else if (id_ready) if_next_state = IF_ADVANCE;
    else if_next_state = IF_SEND;
end
IF_ADVANCE: begin
    if (id_ready) if_next_state = IF_REQ;
    else if_next_state = IF_ADVANCE;
end
default: if_next_state = IF_INIT;
endcase
end

```

3.2 PC 更新逻辑

结合分支预测结果完成预取:预测跳转则加载预测目标地址,预测不跳转则 PC 自增 4;预测错误时直接加载修正后的正确 PC:

```

always @(posedge clk) begin
    if (rst) begin
        pc_reg <= 32'b0;
    end else if (prediction_incorrect) begin
        pc_reg <= pc_correct;
    end else if ((if_state == IF_ADVANCE) && id_ready) begin
        if (predictor_yes_to_if) begin
            pc_reg <= predictor_addr_to_if;
        end else begin
            pc_reg <= pc_reg + 32'd4;
        end
    end
end
assign PC = pc_reg;

```

3.3 总线握手信号

```

assign Inst_Ready = (if_state == IF_WAIT) || (if_state == IF_INIT);
assign Inst_Req_Valid = (if_state == IF_REQ) && ~prediction_incorrect && ~MemRead;

```

```
assign if_to_id_valid = (if_state == IF_SEND) && ~prediction_incorrect;
```

4 ID 译码阶段设计

ID 阶段的设计比较平凡, 和之前的实验项目基本上没有什么的区别, 在这里只提一些流水线改动中比较特别的部分

以下是 ID 阶段工作状态的更新

```
always @(posedge clk) begin
    if (rst) begin
        id_work <= 1'b0;
    end else if (prediction_incorrect) begin
        id_work <= 1'b0;
    end else if (id_ready) begin
        id_work <= if_to_id_valid;
    end
end
assign id_done = ~id_block && ~prediction_incorrect;
assign id_ready = ~id_work || (id_done && ex_ready);
assign id_to_ex_valid = id_done && id_work && ex_ready;
```

在本次实验中, 一个重点就是数据冒险, 我采用的解决方法是数据旁路, 优先使用 EX 阶段运算结果, 其次 MEM 阶段访存结果, 最后 WB 阶段写回结果; 零号寄存器固定为 0, 不参与旁路。

```
wire [31:0] rdata1_true =
    (rs1 == 5'b0) ? 32'b0 :
    (ex_write && (rs1 == ex_waddr_bypass)) ? ex_data_bypass :
    (mem_write_bypass && (rs1 == mem_waddr_bypass)) ? mem_data_bypass :
    (wb_wen_bypass && (rs1 == wb_waddr_bypass)) ? wb_data_bypass :
    rdata1;
```

Load 指令阻塞判断: Load 指令结果仅能在 MEM 阶段生成, 若 ID 阶段源寄存器与 EX/MEM 阶段 Load 指令写地址冲突, 则阻塞 ID 阶段, 等待数据就绪:

```
wire ex_related = ex_write && ((rs1 == ex_waddr_bypass) || (rs2 == ex_waddr_bypass));
wire mem_related = mem_write_bypass && ((rs1 == mem_waddr_bypass) || (rs2 == mem_waddr_bypass));
wire id_block = (ex_related && ex_load) || (mem_related && mem_load_bypass);
```

5 EX 执行阶段设计

EX 阶段的设计也同样比较平凡, 值得一提的就是对于分支预测阶段的处理, 代码段如下

```
wire ex_branch_check =
    (ex_func == 3'b000) ? alu_zero :
    (ex_func == 3'b001) ? ~alu_zero :
    ((ex_func == 3'b100) || (ex_func == 3'b110)) ? ~alu_zero :
    ((ex_func == 3'b101) || (ex_func == 3'b111)) ? alu_zero :
    1'b0;
```

```

// 最终真实分支结果：是分支指令 且 条件满足
wire ex_branch_real = ex_branch_check && ex_branch;

reg ex_block_cancel;
always @(posedge clk) begin
    if (rst) begin
        ex_block_cancel <= 1'b1;
    end else if (id_to_ex_valid) begin // 新指令进入EX阶段，打开检测
        ex_block_cancel <= 1'b0;
    end else begin
        ex_block_cancel <= 1'b1;
    end
end
// 预测结果 != 真实结果 → 预测错误
assign prediction_incorrect = (ex_branch_real ^ ex_prediction) && ~ex_block_cancel;

```

6 MEM 阶段设计

比较值得一提的是设计了 SL 这个阶段，用于判断是读指令还是写指令，不过这个阶段貌似也没有什么作用，可以直接从 SL_BEFORE 跳转至对应的 RDW 和 SL_DONE 阶段，来进行时间开销下的优化，此外 MEM 阶段和之前的也比较类似，在此不多赘述

```

always @(*) begin
    case (mem_state)
        MEM_INIT: mem_next_state = MEM_SL_BEFORE;
        MEM_SL_BEFORE: begin
            if (mem_work) begin
                if (mem_load || mem_store) begin
                    mem_next_state = MEM_SL;
                end else begin
                    mem_next_state = MEM_SL_DONE;
                end
            end else begin
                mem_next_state = MEM_SL_BEFORE;
            end
        end
        MEM_SL: begin
            if (mem_load && Mem_Req_Ready) begin
                mem_next_state = MEM_RDW;
            end else if (mem_store && Mem_Req_Ready) begin
                mem_next_state = MEM_SL_DONE;
            end else begin
                mem_next_state = MEM_SL;
            end
        end
        MEM_RDW: begin
            if (Read_data_Valid && Read_data_Ready) begin
                mem_next_state = MEM_SL_DONE;
            end
        end
    endcase
end

```



```

        end else begin
            mem_next_state = MEM_RDW;
        end
    end
MEM_SL_DONE: begin
    if (ex_to_mem_valid) begin
        mem_next_state = MEM_SL_BEFORE;
    end else begin
        mem_next_state = MEM_SL_DONE;
    end
end
default: mem_next_state = MEM_INIT;
endcase
end

```

7 WB 阶段设计

WB 阶段的设计同样也比较简单, 主要的用处就是将数据写回给寄存器堆 (寄存器堆在 ID 阶段实例化), 此外还有一个点就是 inst_retire 的产生, 代码段如下

```

wire inst_retire_valid = wb_work;
wire inst_wen = inst_retire_valid && wb_rf_wen;
assign inst_retire = {inst_wen, wb_rf_waddr, wb_rf_wdata, wb_pc};

```

三、 学习总结 (总结设计、开发与调试过程中遇到的问题, 及实验中的收获与不足)

1 问题总结 (实验过程中遇到的问题、对问题的思考过程及解决方法)

1. 流水线无法正确流动。由于在本次实验中, 流水线的运行实际上非常依赖于握手信号的产生, 如果不能正确的产生握手信号, 那么流水线就会一直卡在握手的环节, 导致一直一直等待, 无法进行, 所以这种 bug 除了干瞪眼以外, 首先就是可以利用改变 sim_main.cpp 中 TIME_LIMIT 宏, 来确定是否是产生死循环或者一直等待, 之后就是利用 emu 加速仿真的, 改变仿真的时钟周期数, 之后观察波形来检测哪里的握手信号一直悬空来 debug

2. 分支预测导致的一直阻塞。在调试过程中, 同样有一种一直卡住的问题, 就是分支预测错误后的冲刷, 因为我们实际上需要的 prediction_incorrect 是一个脉冲信号, 而非一直拉高, 如果没有一个信号来控制它的位置, 那么整个流水线就会一直停留在阻塞的状态不继续向下进行, 于是我利用了 block_cancel 来进行阻塞的取消. 这样一个周期之后 ID 和 IF 阶段就会有正确的结果, 之后恢复流水线的运作

3. 我再调试过程中, 发现了一个问题就是面对 LUI 和 AUIPC 指令, 虽然我加入了数据旁路, 但是并不能正确地写回正确的值, 后来经过调试我发现, 落后了时序, 导致我的数据旁路并不能取出正确的数据, 于是后来我将这几个命令的写回结果的产生前提到 EX 阶段的 Result 中, 这样就能在最早的产生的时候就能取出正确的结果。

2 收获总结 (在数字电路设计方法、RTL 编写方法、数字电路验证方法及其他方面的收获)

在本实验中, 教师团队引入了报错分析, 帮助很多, 更快地帮助我定位并解决了代码问题。

序号	问题类别	出错模块	问题现象	产生原因	解决办法
1	流水线逻辑	IF	流水线停滞	握手信号与时序逻辑不匹配	规范阶段握手逻辑
2	分支预测	EX	分支预测判断出错	分支条件判断代码逻辑有误	修正判断逻辑
3	时序逻辑	全流水阶段	预测错误引发流水线卡死	冲刷信号无周期限制	新增时序寄存器
4	数据通路	ID	数据旁路取值错误	特殊指令结果生成位置靠后	前移运算结果生成至 EX
5	状态设计	IF 段	取指效率低	握手与冲刷时序冲突	增加同步控制信号

表 1: 实验问题总结

类别	名称	体会	后续计划
对 cpu 的理解	理解多周期 cpu	更加熟悉 cpu 运行原理,即几个阶段的整合	多复习讲义
代码编写	编写 verilog 并实现 always 逻辑	代码编写更加熟练	编写代码
debug 能力	寻找错误	寻找代码错误原因更加熟练	辅助工具实现 debug

表 2: 收获总结

四、 实验时长

在课后,你花费了大约 20 小时完成此次实验,其中各个阶段的用时如表 3所示。

步骤	时长(小时)	说明
设计	6	认真学习课程内容,理解所需实现的板块
开发	6	完成实验代码的编写等
调试	8	进行测试样例的测试,不断修正错误

表 3: 实验时长